Parameterized queries are not vulnerable to SQL injection attacks, so the attack presented earlier won't work. If you use EF Core or other ORMs to access data using standard LINQ queries, you won't be vulnerable to injection attacks. EF Core automatically creates all SQL queries using parameterized queries to protect you. Even if you're using the low-level ADO.NET database APIs, stick to parameterized queries!

NOTE I've talked about SQL injection attacks only in terms of a relational database, but this vulnerability can appear in NoSQL and document databases too. Always use parameterized queries or the equivalent, and don't craft queries by concatenating strings with user input.

Injection attacks have been the number-one vulnerability on the web for more than a decade, so it's crucial to be aware of them and how they arise. Whenever you need to write raw SQL queries, make sure that you always use parameterized queries.

The next vulnerability is also related to attackers accessing data they shouldn't be able to. It's a little subtler than a direct injection attack but is trivial to perform; the only skill the attacker needs is the ability to count.

29.4.3 Preventing insecure direct object references

Insecure direct object reference is a bit of a mouthful, but it means users accessing things they shouldn't by noticing patterns in URLs. Let's revisit our old friend the recipe app. As a reminder, the app shows you a list of recipes. You can view any of them, but you can edit only recipes you created yourself. When you view someone else's recipe, there's no Edit button visible.

A user clicks the Edit button on one of their recipes and notices that the URL is `/Recipes/Edit/120`. That `120` is a dead giveaway as being the